

MAT21-6

Graph Theory

Albert School — 18 hours

Preface

You arrive at this course already fluent in the two languages it speaks. From **Linear Algebra I and II** (MAT11-3, MAT21-1) you can multiply matrices, solve $AX = B$, and — most importantly here — read a square matrix as an object whose **powers** and **eigenvectors** encode long-run behaviour. From **Descriptive Statistics** (MAT11-5) you have met the discipline of turning messy real data into a structured object you can reason about. Graph theory sits exactly at this junction.

A **graph** is the mathematician’s model of a network: dots (**vertices**) joined by links (**edges**). Friendships, road maps, web pages and their hyperlinks, molecules, task dependencies, the flight routes of an airline — all are graphs. The subject has two faces, and this course insists on holding both at once. The **combinatorial** face is the picture: you can see a path, a cycle, a connected blob. The **algebraic** face is the matrix: every graph is also an **adjacency matrix** A , and once it is, the tools of Linear Algebra come flooding in. The single most important slogan of the course is that

the number of walks of length k from vertex i to vertex j is the entry $(A^k)_{ij}$ —

a combinatorial question answered by a matrix power. This is the bridge MAT21-1 promised when it introduced graph adjacency matrices, and we cross it in full here.

We proceed in six movements. First the **foundations** — the vocabulary of graphs and the one theorem (the **handshake lemma**) that every degree count must obey. Then **trees**, the most important special shape, characterized three equivalent ways. Next the **matrix representations** that let Linear Algebra in, culminating in the **Laplacian** $L = D - A$, a symmetric positive-semidefinite matrix that is the gateway to spectral graph theory. We then become algorithmic: **traversals** (BFS and DFS), the **bipartite** test by 2-colouring, and shortest paths in unweighted graphs. We add weights and meet **Dijkstra** and **Kruskal**, the two classic greedy algorithms, and the **priority queue** that powers them. We finish at the frontier where this all became a billion-dollar idea: **PageRank**, which models the web as a graph, a surfer as a **random walk**, and importance as the **dominant eigenvector** of a stochastic matrix — the exact Markov-chain machinery of MAT21-1, now applied to a real network.

Keep the slogan in view: **draw it to see it, matricize it to compute it.**

Contents

Preface	2
1 Graph foundations	4
1.1 A first example: a small social network	4
1.2 Definitions	4
1.3 Connectivity and components	5
1.4 The handshake lemma	5
2 Trees	6
2.1 A first example: the same tree, three descriptions	6
2.2 Three equivalent characterizations	6
2.3 Shortest paths on a tree	7
3 Matrix representations and the Laplacian	8
3.1 The adjacency matrix	8
3.2 Counting walks: the headline theorem	8
3.3 The incidence and degree matrices	9

3.4	The graph Laplacian	9
4	Traversal and structure	11
4.1	BFS and DFS: two ways to fan out	11
4.2	Connectivity by traversal	12
4.3	Shortest paths in unweighted graphs	12
4.4	Bipartite graphs and 2-colouring	12
5	Weighted graphs: shortest paths and spanning trees	13
5.1	Dijkstra's shortest-path algorithm	14
5.2	Minimum spanning trees: Kruskal's algorithm	15
6	Random walks and PageRank	16
6.1	Random walks and the transition matrix	16
6.2	PageRank as a fixed point	16
6.3	Power iteration by hand	17

1 Graph foundations

A graph records **which things are related to which**. The things are vertices; a relation between two of them is an edge. Everything else in this chapter — degree, path, cycle, connectivity — is built from this single primitive.

1.1 A first example: a small social network

Five people; an edge means “these two are friends”. Picture it:

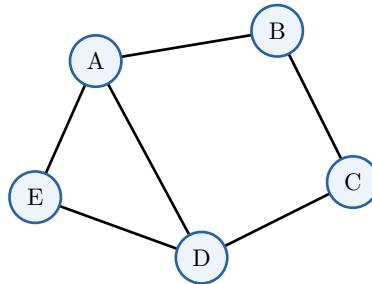


Figure 1: A simple undirected graph on five vertices $\{A, B, C, D, E\}$ with six edges. A is friends with B , E , and D , so $\deg(A) = 3$. There is a path $E - A - B - C$ of length 3, and a cycle $A - D - E - A$ of length 3.

From this drawing you can already read off the notions we now name precisely: A touches three edges (its **degree** is 3); you can travel $E - A - B - C$ (a **path**); you can leave A and return, $A - D - E - A$ (a **cycle**); and starting anywhere you can reach everyone else (the graph is **connected**).

1.2 Definitions

Definition 1 A (**simple, undirected**) **graph** is a pair $G = (V, E)$ where V is a finite set of **vertices** and E is a set of **edges**, each edge an unordered pair $\{u, v\}$ of distinct vertices. “Simple” means: no edge from a vertex to itself (no **loops**) and at most one edge between any two vertices (no **multi-edges**). We write $n = |V|$ for the number of vertices and $m = |E|$ for the number of edges.

Two variants recur throughout the course.

Definition 2

- A **directed graph** (digraph) replaces unordered pairs by **ordered** pairs: an edge (u, v) points **from** u **to** v , and (u, v) and (v, u) are different.
- A **weighted graph** attaches a number $w(e)$ — a length, cost, or capacity — to each edge e .

Definition 3 The **degree** $\deg(v)$ of a vertex v in an undirected graph is the number of edges incident to it. In a directed graph one distinguishes the **in-degree** $\deg^-(v)$ (edges pointing into v) from the **out-degree** $\deg^+(v)$ (edges pointing out of v).

Definition 4 A **walk** is a sequence of vertices $v_0, v_1, \dots, v_k$ with each consecutive pair $\{v_i, v_{i+1}\}$ an edge; its **length** is the number of edges k . A **path** is a walk with no repeated

vertex. A **cycle** is a walk of length ≥ 3 that starts and ends at the same vertex and otherwise repeats no vertex.

Common pitfall Keep **walk**, **path**, and **cycle** distinct — they are confused constantly. A **walk** may revisit vertices and re-use edges freely ($A - D - A - B$ is a walk). A **path** may not repeat any vertex. When we count “walks of length k ” with matrix powers in Chapter 3, repetition is **allowed** — that is precisely why the count is a clean matrix entry rather than a hard combinatorial quantity.

1.3 Connectivity and components

Definition 5 Two vertices u, v are **connected** if there is a walk (equivalently, a path) between them. A graph is **connected** if every pair of vertices is connected. In general, “connected to” partitions V into **connected components** — the maximal sets of mutually reachable vertices.

The graph above has a single component. If we deleted the edge $A - B$ and the edge $B - C$, vertex B would float free as its own component, and the graph would have two components.

1.4 The handshake lemma

Here is the first theorem of the subject, and a perfect example of the algebraic discipline this course demands: a statement about a picture, proved by counting the same thing two ways.

Theorem 6 (Handshake lemma) In any undirected graph $G = (V, E)$,

$$\sum_{v \in V} \deg(v) = 2|E|.$$

Proof. Count the set of **incidences** — pairs (v, e) where vertex v is an endpoint of edge e — in two ways. Grouping by vertex, each v contributes exactly $\deg(v)$ incidences, for a total of $\sum_v \deg(v)$. Grouping by edge, each edge has exactly two endpoints, contributing 2 incidences, for a total of $2|E|$. The two counts are equal. $\square$

The name comes from a party: if every guest shakes some hands, the total number of hands shaken (summed over people) is twice the number of handshakes, because each handshake involves two hands.

Corollary 7 In every graph the number of vertices of **odd** degree is even.

Proof. Split the sum: $\sum_v \deg(v) = \sum_{\text{even-deg}} \deg(v) + \sum_{\text{odd-deg}} \deg(v)$. The left side is $2|E|$, even; the first right-hand sum is even (a sum of even numbers). So $\sum_{\text{odd-deg}} \deg(v)$ is even — but it is a sum of odd numbers, which is even only if there is an **even number** of them. $\square$

The lemma’s everyday use is as a **filter** that rules out impossible graphs before you waste time trying to draw them.

Worked example — ruling out an impossible degree sequence. Can a simple graph have degree sequence $(3, 3, 3, 3, 3)$ — five vertices each of degree 3? The degree sum is $5 \cdot 3 = 15$, which is **odd**. By the handshake lemma the degree sum must equal $2|E|$, an even number. No such graph exists — equivalently, there are five vertices of odd degree, and five is odd, contradicting the corollary. (Compare $(3, 3, 3, 3)$ on four vertices: sum $12 = 2 \cdot 6$, possible — it is the complete graph K_4 .)

Common pitfall The handshake lemma is **necessary, not sufficient**. An even degree sum does not guarantee a graph exists with that sequence: $(4, 1, 1)$ sums to 6 but is impossible, because a vertex of degree 4 needs four distinct neighbours and only two others are available. Use the lemma to **reject**; existence needs a construction or a finer criterion.

Exercises

1. For the five-vertex graph in the opening figure, list every vertex's degree and verify the handshake lemma.
2. A graph has 10 vertices each of degree 4. How many edges does it have?
3. Explain why every simple graph on $n \geq 2$ vertices has two vertices of the same degree. (Hint: the possible degrees are $0, \dots, n-1$, but 0 and $n-1$ cannot both occur.)
4. In a directed graph, prove $\sum_v \deg^+(v) = \sum_v \deg^-(v) = |E|$.

2 Trees

Among all connected graphs, **trees** are the leanest: connected, but with not a single edge to spare. They are the shape of hierarchies, of file systems, of the traversal structures we build in Chapter 4, and of the minimum spanning trees of Chapter 5. Remarkably, three quite different-sounding descriptions pick out exactly the same graphs.

2.1 A first example: the same tree, three descriptions

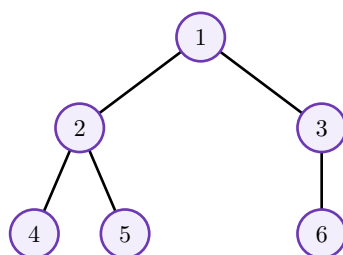


Figure 2: A tree on $n = 6$ vertices. It has exactly $m = 5 = n - 1$ edges, it is connected, and it contains no cycle. Between any two vertices — say 4 and 6 — there is exactly one path: $4 - 2 - 1 - 3 - 6$.

Count the edges of that picture: six vertices, five edges. Try to find a cycle: there is none. Try to disconnect it by deleting any single edge: you always can. Each of these is a **defining** property of a tree, and the next theorem says they travel together.

2.2 Three equivalent characterizations

Definition 8 A **tree** is a connected graph with no cycles. A graph whose every component is a tree (no cycles, but possibly disconnected) is a **forest**. A **leaf** is a vertex of degree 1.

Theorem 9 (Characterizations of trees) Let G be a graph on n vertices. The following are equivalent.

1. G is **connected and acyclic** (a tree, by definition).
2. G is **connected and has exactly $n - 1$ edges**.
3. Between every pair of vertices there is a **unique path**.

Proof. (1 $\Rightarrow$ 3.) In a connected graph every pair is joined by at least one path. If two distinct paths joined some pair u, v , following one out and the other back would trace a closed walk containing a cycle — contradicting acyclicity. So the path is unique.

(3 $\Rightarrow$ 1.) Unique paths force connectedness (at least one path exists for each pair). And a cycle would give two distinct paths between any two of its vertices, contradicting uniqueness; so G is acyclic.

(1 $\Leftrightarrow$ 2.) We show a connected graph is acyclic iff it has $n - 1$ edges. A connected graph on n vertices needs at least $n - 1$ edges (each new vertex added to a connected piece needs ≥ 1 new edge). If it has exactly $n - 1$, no edge is spare, so removing any edge disconnects it — no edge lies on a cycle, and G is acyclic. Conversely a connected acyclic graph, built by adding its vertices one at a time along the unique-path structure, uses exactly one edge per vertex after the first, i.e. $n - 1$ edges. $\square$

Common pitfall All three conditions in each characterization are needed together. “ $n - 1$ edges” **alone** does not make a tree: a triangle plus an isolated vertex has 4 vertices and $3 = n - 1$ edges but is neither connected nor acyclic. You must pair the edge count with **connectedness** (or with acyclicity — “acyclic with $n - 1$ edges” also forces a tree). Any **two** of {connected, acyclic, $n - 1$ edges} imply the third.

2.3 Shortest paths on a tree

Trees make the shortest-path problem trivial, and this is worth isolating because the general algorithms of later chapters all reduce to “build a tree, read the path”.

Proposition 10 In a tree, the unique path between two vertices **is** the shortest path between them — there is no other path to compete with it. Its length is the number of edges along it (the **tree distance**).

So to find the shortest route between vertices u and v in a tree, you simply trace the one path that exists. In the tree above, the distance from 4 to 6 is 4, along $4 - 2 - 1 - 3 - 6$, and no shorter route can exist because no other route exists at all. This is the degenerate, cycle-free case of the BFS and Dijkstra computations to come: those algorithms spend all their effort precisely because general graphs offer **many** competing paths, which trees never do.

Exercises

5. Draw all distinct trees on 4 vertices (up to relabelling). How many are there?
6. A tree has $n = 12$ vertices. How many edges does it have? If additionally it has exactly two leaves, what must the tree look like?
7. Prove that every tree with at least two vertices has at least two leaves. (Hint: consider a longest path and its endpoints.)

8. Show that adding any single edge to a tree creates exactly one cycle.

3 Matrix representations and the Laplacian

We now matricize. Encoding a graph as a matrix is not mere bookkeeping: it is the move that lets the entire apparatus of Linear Algebra — matrix products, eigenvalues, positive-definiteness — speak about graphs. This chapter reactivates MAT11-3 and MAT21-1 directly.

3.1 The adjacency matrix

Definition 11 Fix an ordering $v_1, \dots, v_n$ of the vertices. The **adjacency matrix** $A \in M_n(\mathbb{R})$ has entries

$$A_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge,} \\ 0 & \text{otherwise.} \end{cases}$$

For a **weighted** graph one stores the weight $w(v_i, v_j)$ in place of 1; for a **directed** graph $A_{ij} = 1$ records the edge $v_i \rightarrow v_j$ only.

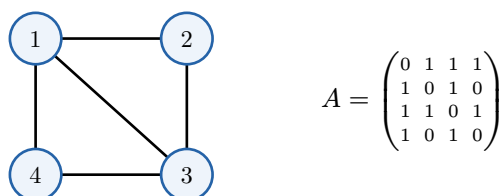


Figure 3: A graph and its adjacency matrix. A is **symmetric** ($A = A^T$) because the graph is undirected. Row i sums to $\deg(v_i)$: row 1 sums to $3 = \deg(1)$. The diagonal is 0 because the graph is simple (no loops).

Two facts are immediate and worth stating, since every later computation leans on them.

Proposition 12 For the adjacency matrix of a simple undirected graph:

1. A is **symmetric**: $A_{ij} = A_{ji}$, so $A = A^T$.
2. The i -th **row sum** (equivalently column sum) equals $\deg(v_i)$, and the diagonal entries A_{ii} are all 0.

Reading information **off** the matrix is exactly the inverse skill of building it: given A , the degree of vertex i is the sum of row i ; the edge weights are the off-diagonal entries; symmetry tells you the graph is undirected.

3.2 Counting walks: the headline theorem

Here is the theorem that justifies the whole algebraic detour, the one promised in the preface and first met in MAT21-1.

Theorem 13 (Walks via matrix powers) Let A be the adjacency matrix of a graph. For every integer $k \geq 1$, the entry

$$(A^k)_{ij} = \text{the number of walks of length } k \text{ from } v_i \text{ to } v_j.$$

Proof. Induct on k . For $k = 1$, $(A^1)_{ij} = A_{ij}$ counts the length-1 walks (single edges) from i to j — the definition. Assume the claim for k . A walk of length $k + 1$ from i to j consists of a walk of length k from i to some intermediate vertex ℓ , followed by a single edge $\ell \rightarrow j$. Summing over all choices of ℓ ,

$$(A^{k+1})_{ij} = \sum_{\ell=1}^n (A^k)_{i\ell} A_{\ell j},$$

which is exactly the rule for the matrix product $A^k \cdot A$. So the count for $k + 1$ equals $(A^{k+1})_{ij}$. $\square$

The proof reveals **why** matrix multiplication — that strange row-times-column rule from MAT11-3 — is the right operation: multiplying A^k by A “concatenates a walk of length k with one more edge”, and the summation over the shared index ℓ is the summation over the intermediate vertex.

Worked example — counting length-2 walks. For the square-with-diagonal graph above,

$$A^2 = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}^2 = \begin{pmatrix} 3 & 1 & 2 & 1 \\ 1 & 2 & 1 & 2 \\ 2 & 1 & 3 & 1 \\ 1 & 2 & 1 & 2 \end{pmatrix}.$$

Read off $(A^2)_{11} = 3$: there are three length-2 walks from vertex 1 back to itself, namely $1 \rightarrow 2 \rightarrow 1$, $1 \rightarrow 3 \rightarrow 1$, $1 \rightarrow 4 \rightarrow 1$ — one for each neighbour, so $(A^2)_{ii} = \deg(v_i)$ always. And $(A^2)_{13} = 2$: the two walks $1 \rightarrow 2 \rightarrow 3$ and $1 \rightarrow 4 \rightarrow 3$.

Common pitfall $(A^k)_{ij}$ counts **walks**, which may repeat vertices and edges — not paths. The walk $1 \rightarrow 2 \rightarrow 1$ is counted even though it backtracks. Do not read $(A^2)_{11} = 3$ as “three paths”; there are no length-2 paths from a vertex to itself. Matrix powers are blind to repetition, which is what makes them computable.

3.3 The incidence and degree matrices

Two more matrices complete the toolkit, and combine into the star of the chapter.

Definition 14

- The **degree matrix** D is the diagonal matrix with $D_{ii} = \deg(v_i)$ and zeros off the diagonal.
- The (unsigned) **incidence matrix** of a graph with n vertices and m edges is the $n \times m$ matrix B with $B_{ie} = 1$ if vertex i is an endpoint of edge e , and 0 otherwise. Each column has exactly two 1’s — the two endpoints of that edge.

The incidence matrix is the matrix face of the handshake lemma: summing all its entries by rows gives $\sum_v \deg(v)$, by columns gives $2m$.

3.4 The graph Laplacian

Combining the degree and adjacency matrices produces the single most important matrix in modern graph theory.

Definition 15 The **graph Laplacian** of an undirected graph is

$$L = D - A,$$

where D is the degree matrix and A the adjacency matrix. Explicitly,

$$L_{ij} = \begin{cases} \deg(v_i) & \text{if } i = j \\ -1 & \text{if } \{v_i, v_j\} \text{ is an edge,} \\ 0 & \text{otherwise.} \end{cases}$$

Worked example — a Laplacian. For the square-with-diagonal graph, $\deg = (3, 2, 3, 2)$, so

$$D = \begin{pmatrix} 3 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}, \quad L = D - A = \begin{pmatrix} 3 & -1 & -1 & -1 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ -1 & 0 & -1 & 2 \end{pmatrix}.$$

Every row of L sums to 0 — the diagonal degree exactly cancels the -1 's for its neighbours. So $L\mathbf{1} = 0$: the all-ones vector is always an eigenvector of L for the eigenvalue 0.

The two structural properties in the OP follow cleanly, and both reactivate MAT21-1.

Proposition 16 (Symmetry and positive semi-definiteness) The Laplacian $L = D - A$ of an undirected graph is

1. **symmetric**: $L = L^T$; and
2. **positive semi-definite**: $x^T Lx \geq 0$ for every vector $x \in \mathbb{R}^n$. In particular all its eigenvalues are real and ≥ 0 .

Proof. Symmetry. D is diagonal, hence symmetric; A is symmetric because the graph is undirected; so $L = D - A$ is a difference of symmetric matrices, symmetric.

Positive semi-definiteness. The key is the identity, valid for every $x = (x_1, \dots, x_n)$,

$$x^T Lx = \sum_{\{v_i, v_j\} \in E} (x_i - x_j)^2.$$

To see it, expand using the entries of L : the diagonal contributes $\sum_i \deg(v_i)x_i^2$ and the off-diagonal contributes $-2 \sum_{\{i,j\} \in E} x_i x_j$; grouping each edge $\{i, j\}$ gives $x_i^2 - 2x_i x_j + x_j^2 = (x_i - x_j)^2$. As a sum of squares, $x^T Lx \geq 0$ for all x . A symmetric matrix with $x^T Lx \geq 0$ has only real, non-negative eigenvalues (MAT21-1). $\square$

Remark The quadratic form $x^T Lx = \sum_{\text{edges}} (x_i - x_j)^2$ measures how much a labelling x of the vertices “disagrees across edges”. It is zero exactly when x is constant on each connected component — which is why the multiplicity of the eigenvalue 0 of L equals the number of **connected components** of the graph. This is the first whisper of **spectral graph theory**: the eigenvalues of L read off the graph's shape.

Exercises

9. Write the adjacency matrix of the triangle K_3 and compute A^2 and A^3 . Interpret $(A^3)_{11}$.
10. For the path graph $1 - 2 - 3 - 4$, write A , D , and $L = D - A$. Verify that each row of L sums to 0.
11. Using the identity $x^T L x = \sum_{\text{edges}} (x_i - x_j)^2$, evaluate $x^T L x$ for the path graph and $x = (1, 2, 3, 4)$.
12. Show that for any graph the trace of L equals $2|E|$, and explain why via the handshake lemma.

4 Traversal and structure

We turn algorithmic. To **traverse** a graph is to visit its vertices systematically, starting from one and fanning out along edges. The two canonical strategies — **breadth-first** and **depth-first** — are the workhorses behind connectivity tests, bipartiteness checks, and shortest paths in unweighted graphs.

4.1 BFS and DFS: two ways to fan out

Both algorithms start at a **source** vertex s , mark vertices as visited, and grow a **traversal tree** whose edges are the ones first used to reach each new vertex. They differ only in the **order** in which waiting vertices are explored.

Definition 17

- **Breadth-first search (BFS)** explores in **layers**: first s , then all neighbours of s , then all their unvisited neighbours, and so on. It uses a **queue** (first-in, first-out): newly discovered vertices wait at the back of the line.
- **Depth-first search (DFS)** plunges as **deep** as possible before backing up: from the current vertex it walks to an unvisited neighbour, repeats, and backtracks only when stuck. It uses a **stack** (last-in, first-out), often via recursion.

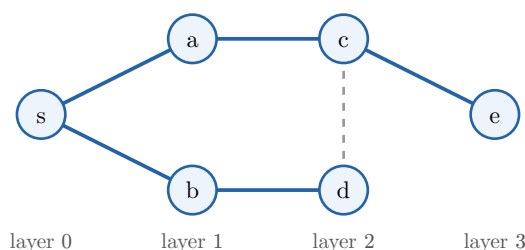


Figure 4: A BFS from source s . Blue edges form the **BFS tree**; the dashed edge $c - d$ is a non-tree edge joining the same layer. BFS discovers vertices in order of distance from s : s (layer 0); a, b (layer 1); c, d (layer 2); e (layer 3). Each vertex's layer number is its shortest-path distance from s .

Worked example — BFS and DFS by hand. Run both on the graph above from s , exploring neighbours in alphabetical order.

BFS (queue). Visit s ; enqueue a, b . Dequeue a , enqueue c . Dequeue b , enqueue d . Dequeue c , enqueue e (and see d already queued). Dequeue d , e . Order: s, a, b, c, d, e — by layers.

DFS (stack/recursion). Visit s , go to a , then c , then d (via $c - d$), then back has nowhere new from d except none, backtrack to c , go to e ; backtrack fully, then b already visited. Order: s, a, c, d, e, b — one deep plunge before breadth.

4.2 Connectivity by traversal

A single traversal from s visits **exactly** the connected component of s — every vertex reachable from s , and no others. This gives the simplest possible connectivity test.

Proposition 18 A graph is connected if and only if a single BFS or DFS from any vertex visits all n vertices. Repeatedly launching a fresh traversal from an unvisited vertex, until none remain, discovers all connected components — one per launch.

4.3 Shortest paths in unweighted graphs

BFS does more than find a path; it finds the **shortest** one, and this is the property that makes it indispensable.

Theorem 19 (BFS computes unweighted shortest paths) In BFS from a source s , every vertex v is discovered in **layer** $d(v)$, and $d(v)$ equals the length of a shortest path (fewest edges) from s to v . The BFS tree path from s to v realizes that shortest distance.

Proof. We argue that the layer index equals the shortest-path distance, by induction on the layer. Layer 0 is s , at distance 0. Suppose every vertex in layers $0, \dots, k$ has been assigned its true distance. A vertex v first discovered in layer $k + 1$ is adjacent to some layer- k vertex u , giving a path of length $k + 1$, so $d(s, v) \leq k + 1$. It cannot be shorter: if $d(s, v) \leq k$, then v would have a neighbour at distance $\leq k - 1$, hence (inductive hypothesis) in a layer $\leq k - 1$, and BFS would have discovered v no later than layer k — contradicting “first discovered in layer $k + 1$ ”. So $d(s, v) = k + 1$. $\square$

The crux is that BFS’s queue processes vertices **in non-decreasing order of distance**: it completely empties layer k before touching layer $k + 1$. This is exactly the property that **fails** once edges carry different weights — which is why weighted shortest paths need Dijkstra, not BFS.

Common pitfall BFS gives shortest paths only when **every edge counts the same** (unweighted, or all weights equal). On a weighted graph the fewest-edges route can be far **longer** in total weight than a many-edge route. Do not run BFS on a weighted graph and expect a shortest path; that is Dijkstra’s job (next chapter). DFS does **not** give shortest paths even in the unweighted case — its deep plunges overshoot.

4.4 Bipartite graphs and 2-colouring

A traversal also decides a fundamental structural question: can the vertices be split into two camps with all edges crossing between them?

Definition 20 A graph is **bipartite** if its vertices can be partitioned into two sets X and Y so that **every** edge joins a vertex of X to a vertex of Y — equivalently, it can be properly **2-coloured** (each vertex black or white, no edge monochromatic).

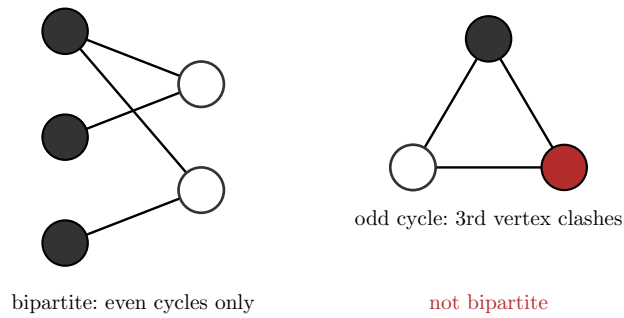


Figure 5: Left: a bipartite graph, properly 2-coloured with black (X) and white (Y); every edge crosses sides. Right: a triangle is an **odd cycle** — colour two vertices black/white, and the third is adjacent to both colours, forcing a clash (red). A graph is bipartite **iff** it has no odd cycle.

Theorem 21 (Bipartite iff no odd cycle) A graph is bipartite if and only if it contains no cycle of **odd** length.

Proof. (Bipartite $\Rightarrow$ no odd cycle.) Walking along any cycle, the colour alternates black, white, black, ... at every step. To **return** to the starting vertex with its original colour, the number of steps must be even. So every cycle has even length.

($\Rightarrow$ via **2-colouring by BFS**.) Conversely, suppose no odd cycle. Run BFS and colour each vertex by the **parity of its layer**: even layers black, odd layers white. A **tree** edge joins consecutive layers (parities differ), so it is properly coloured. A **non-tree** edge joins two vertices whose layers differ by 0 (same layer) — but a same-layer edge would close an odd cycle through the BFS tree, which we excluded. Hence every non-tree edge also joins different layers in parity, the colouring is proper, and G is bipartite. $\square$

This yields a clean **algorithm**: run BFS, 2-colour by layer parity, and check every edge. If any edge is monochromatic, you have found an odd cycle and the graph is not bipartite; otherwise the colouring proves it is.

Exercises

13. Run BFS and DFS on the five-vertex social graph of Chapter 1, from vertex A , exploring neighbours alphabetically. Give the visiting order and the traversal tree for each.
14. Use BFS to find the distance from A to every other vertex in that graph.
15. Is the cycle graph C_6 (a hexagon) bipartite? Is C_5 ? Justify with the odd-cycle criterion and exhibit a 2-colouring where possible.
16. Explain why a tree is always bipartite.

5 Weighted graphs: shortest paths and spanning trees

Real networks carry numbers on their edges — distances, costs, travel times. Two classic **greedy** algorithms dominate weighted graphs: **Dijkstra** finds shortest paths, **Kruskal** builds a minimum spanning tree. Both repeatedly make the locally cheapest choice, and both lean on a **priority queue** to find that cheapest choice fast.

5.1 Dijkstra's shortest-path algorithm

Definition 22 In a weighted graph with edge weights $w(u, v) \geq 0$, the **length** of a path is the **sum** of its edge weights, and the **shortest-path distance** $d(s, v)$ is the minimum length over all s -to- v paths.

Common pitfall Dijkstra's correctness **requires non-negative weights**. With a negative edge, the greedy step — “the closest unfinalized vertex can never be improved later” — fails, because a later detour through a negative edge could undercut a distance already declared final. If negative weights are possible, Dijkstra is the wrong tool. This non-negativity assumption is the single most important precondition to state.

Definition 23 Dijkstra's algorithm from source s maintains a tentative distance $d[v]$ for every vertex ($d[s] = 0$, all others ∞) and a set of **finalized** vertices. Repeat: among non-finalized vertices pick the one u of **smallest** $d[u]$, finalize it, and **relax** each edge out of u —

if $d[u] + w(u, v) < d[v]$ then set $d[v] \leftarrow d[u] + w(u, v)$.

Stop when all reachable vertices are finalized.

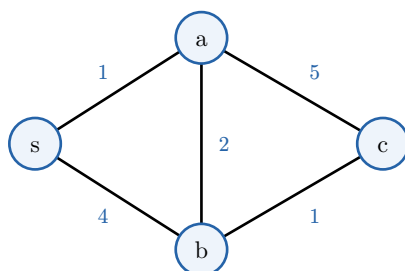


Figure 6: A weighted graph. Dijkstra from s finalizes $s(0)$, then $a(1)$, then $b(3)$ — note b 's direct edge costs 4, but the detour $s - a - b$ costs $1 + 2 = 3$, which is shorter — then $c(4)$ via $s - a - b - c$ ($3 + 1$), beating $s - a - c$ ($1 + 5 = 6$). Distances: $d = (0, 1, 3, 4)$.

Worked example — Dijkstra step by step. On the graph above. Start $d = (0, \infty, \infty, \infty)$ for (s, a, b, c) .

- Finalize s ($d = 0$). Relax: $d[a] = 1$, $d[b] = 4$.
- Smallest non-final is a (1). Finalize. Relax: $d[b] = \min(4, 1 + 2) = 3$; $d[c] = \min(\infty, 1 + 5) = 6$.
- Smallest is b (3). Finalize. Relax: $d[c] = \min(6, 3 + 1) = 4$.
- Finalize c (4). Done: $d = (0, 1, 3, 4)$.

The reason b could be finalized at 3 and never revised is non-negativity: no remaining vertex sits closer than 3, so no future path can re-enter b more cheaply.

Proposition 24 (Why the priority-queue order is correct) When Dijkstra finalizes the non-finalized vertex u of smallest tentative distance, $d[u]$ is already the **true** shortest distance $d(s, u)$.

Proof. Any s -to- u path must, at some point, leave the finalized set for the first time, crossing an edge to a non-finalized vertex x . By the choice of u as the **minimum**, $d[u] \leq d[x]$, and since weights are non-negative the remainder of the path from x to u only adds length. So every s -to- u path has length $\geq d[u]$, meaning $d[u]$ is optimal. This is exactly where **non-negativity** is used. $\square$

The algorithm's efficiency hinges on the repeated request “give me the non-finalized vertex of smallest d ”. A **priority queue** (min-heap) serves each such request — and each distance update — in logarithmic time, instead of scanning all vertices each round. The priority queue is the data structure that makes greedy shortest-path practical.

5.2 Minimum spanning trees: Kruskal's algorithm

A different optimization: connect **all** vertices as cheaply as possible.

Definition 25 A **spanning tree** of a connected graph is a tree (subgraph) that includes every vertex. A **minimum spanning tree (MST)** is a spanning tree whose total edge weight is as small as possible.

Definition 26 **Kruskal's algorithm.** Sort all edges by increasing weight. Scan them in that order, **adding** each edge to the tree-under-construction unless it would **form a cycle** with edges already chosen; skip it if it would. Stop once $n - 1$ edges are chosen — a spanning tree.

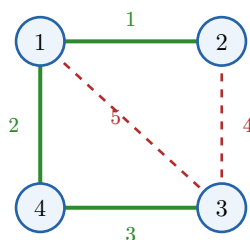


Figure 7: Kruskal on a weighted graph. Edges are taken cheapest-first: weight 1, then 2, then 3 (green, forming the MST). The weight-4 and weight-5 edges (dashed red) are **rejected** because each would close a cycle. Total MST weight $1 + 2 + 3 = 6$; the tree has $n - 1 = 3$ edges.

Worked example — Kruskal step by step. On the four-vertex graph above, edges sorted by weight: $1\{-\}2(1)$, $1\{-\}4(2)$, $3\{-\}4(3)$, $2\{-\}3(4)$, $1\{-\}3(5)$.

- Take $1\{-\}2$ (no cycle). Take $1\{-\}4$ (no cycle). Take $3\{-\}4$ (no cycle) — now 3 edges, all four vertices connected.
- $2\{-\}3$ would close the cycle $2 - 1 - 4 - 3 - 2$: **reject**. $1\{-\}3$ likewise: **reject**.

MST = $\{1\{-\}2, 1\{-\}4, 3\{-\}4\}$, total weight 6.

The cycle test — “does adding this edge join two vertices already connected?” — is itself a connectivity query, efficiently handled by a **union-find** structure; conceptually it is the tree characterization of Chapter 2 in action (“adding an edge within a connected piece creates a cycle”). As with Dijkstra, sorting and the priority-style selection of the next cheapest edge is where the computational work lives.

Remark Both algorithms are **greedy** — they commit to the locally cheapest option and never reconsider — yet both are provably **globally optimal**. This is unusual and special to the structure of shortest paths and spanning trees; greed fails for most optimization problems, so do not over-generalize the lesson.

Exercises

17. Run Dijkstra from s on a graph of your choice with five vertices and record the order of finalization and the final distances.
18. Give a small example with one negative-weight edge where Dijkstra returns a wrong distance, and explain which finalization step goes wrong.
19. Run Kruskal on the graph with vertices $\{1, 2, 3, 4\}$ and edges $1\{-\}2(4), 2\{-\}3(1), 3\{-\}4(2), 4\{-\}1(3), 1\{-\}3(5)$. Give the MST and its weight.
20. Explain why an MST of a connected graph on n vertices always has exactly $n - 1$ edges, citing the tree characterization of Chapter 2.

6 Random walks and PageRank

We close at the meeting point of every thread in the course: graphs, matrix powers, stochastic matrices, and the dominant eigenvector of MAT21-1 — assembled into **PageRank**, the algorithm that originally ranked the web. The level here is conceptual-computational: understand the model and run a step or two by hand.

6.1 Random walks and the transition matrix

Imagine a walker on a graph who, at each step, moves to a **uniformly random neighbour** of the current vertex. This is a **random walk**, and it is governed by a matrix.

Definition 27 The **transition matrix** of a random walk on a directed graph has entry

$$P_{ij} = \begin{cases} 1/\deg^+(j) & \text{if there is an edge } j \rightarrow i \\ 0 & \text{otherwise,} \end{cases}$$

the probability of stepping to i from j . Each **column** sums to 1 (a walker at j goes somewhere), so P is **column-stochastic** — exactly the object from the Markov-chain section of MAT21-1.

If x_k is the probability distribution over vertices at step k (a vector of non-negative entries summing to 1), then one step advances it by

$$x_{k+1} = Px_k,$$

and k steps give $x_k = P^k x_0$. We are back to **matrix powers** — now of a stochastic matrix, where the long-run behaviour is the eigenvector story of MAT21-1.

6.2 PageRank as a fixed point

The PageRank idea: a web page is **important** if important pages link to it. This is circular, and the circularity is resolved exactly as a **fixed point** — a distribution unchanged by one more step of the walk.

Definition 28 Model the web as a directed graph (pages = vertices, hyperlinks = directed edges), with column-stochastic transition matrix P . The **PageRank vector** π is the **stationary distribution** of the walk:

$$P\pi = \pi, \quad \sum_i \pi_i = 1, \quad \pi_i \geq 0.$$

Equivalently, π is an **eigenvector of P for the eigenvalue 1** — the dominant eigenvalue of a stochastic matrix — normalized to a probability distribution. A page's rank π_i is the long-run fraction of time the random surfer spends on it.

This is the precise statement promised by MAT21-1's Markov-chain material: every column-stochastic matrix has 1 as an eigenvalue, and the corresponding eigenvector, normalized, is the stationary distribution to which $x_k = P^k x_0$ converges.

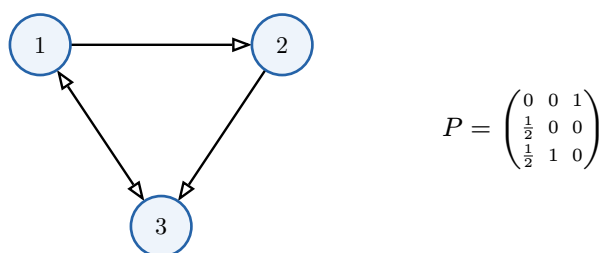


Figure 8: A tiny web of three pages. Page 1 links to 2 and 3 (out-degree 2, so each outgoing probability is $\frac{1}{2}$); page 2 links to 3; pages 1 and 3 link to each other (the double-headed arrow). The column-stochastic matrix P has $P_{ij} = 1/\deg^+(j)$. Its eigenvector for eigenvalue 1, normalized, is the PageRank vector.

6.3 Power iteration by hand

We rarely solve $P\pi = \pi$ directly; instead we **iterate**. Starting from the uniform distribution and repeatedly applying P , the distribution converges to π — this is **power iteration**, the dominant-eigenvector method of MAT21-1 in action.

Worked example — two steps of power iteration. For the three-page web above, start uniform $x_0 = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})^T$.

$$x_1 = Px_0 = \begin{pmatrix} 0 & 0 & 1 \\ \frac{1}{2} & 0 & 0 \\ \frac{1}{2} & 1 & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{3} \\ \frac{1}{3} \\ \frac{1}{3} \end{pmatrix} = \begin{pmatrix} \frac{1}{3} \\ \frac{1}{6} \\ \frac{1}{2} \end{pmatrix}.$$

Apply P again:

$$x_2 = Px_1 = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{6} \\ \frac{1}{3} \end{pmatrix}.$$

Each x_k stays a probability distribution (entries ≥ 0 , summing to 1), and the iterates march toward the stationary π . Solving $P\pi = \pi$ exactly gives $\pi = (\frac{2}{5}, \frac{1}{5}, \frac{2}{5})^T$: pages 1 and 3 tie as most important, each linked to from a page that is itself well-linked.

Common pitfall Power iteration converges to the stationary distribution under the conditions met in MAT21-1: the chain must be such that 1 is a **dominant** eigenvalue (every other eigenvalue strictly smaller in absolute value). Real PageRank enforces this by the **damping** trick — with small probability the surfer “teleports” to a uniformly random page — which also fixes **dangling pages** (no outgoing links, an all-zero column). We note this only conceptually; the linear-algebra core is the eigenvector-for-1 above.

Remark Step back and see the whole arc. A **graph** (Chapter 1) became an **adjacency matrix** (Chapter 3); its powers **counted walks**; normalizing those steps gave a **stochastic matrix** whose **dominant eigenvector** (MAT21-1) is **importance**. The picture posed the question — who matters in this network? — and linear algebra answered it. That is the discipline this course exists to teach.

Exercises

21. Write the transition matrix P for a random walk on the undirected triangle K_3 and find its stationary distribution by symmetry.
22. For the three-page web, carry power iteration one more step to compute x_3 and compare with $\pi = (\frac{2}{5}, \frac{1}{5}, \frac{2}{5})^T$.
23. A page with no outgoing links gives an all-zero column, so P is not stochastic. Explain in one sentence why teleportation (damping) repairs this.
24. Verify directly that $\pi = (\frac{2}{5}, \frac{1}{5}, \frac{2}{5})^T$ satisfies $P\pi = \pi$ for the three-page web.

The course in one breath. A **graph** $G = (V, E)$ models a network; **degrees** obey the **handshake lemma** $\sum \deg = 2|E|$. **Trees** are connected and acyclic — equivalently $n - 1$ edges, equivalently unique paths. Encoding G as its **adjacency matrix** A lets Linear Algebra in: $(A^k)_{ij}$ **counts walks**, and the **Laplacian** $L = D - A$ is symmetric and positive semi-definite, its spectrum reading off connectivity. **BFS** fans out in layers (giving unweighted shortest paths and a 2-colouring test for **bipartiteness** — no odd cycles), **DFS** plunges deep. With weights, **Dijkstra** (non-negative weights!) finds shortest paths and **Kruskal** builds a minimum spanning tree, both greedy, both powered by a **priority queue**. Finally a **random walk** turns G into a **stochastic matrix** P , and **PageRank** is its stationary distribution — the **dominant eigenvector** of MAT21-1, computed by **power iteration**. Draw it to see it; matricize it to compute it.